

SISTEMAS DE COMPUTADORES – 2021/2022 2ºSemestre
Exame 28/06/2022

Sem consulta
Duração 50 min

Nome: _____ Nº _____

Folha de Respostas

NOTAS:

1 – Em todas as questões de escolha múltipla deverá assinalar uma resposta correta, se a resposta assinalada for incorreta sofrerá uma penalização até 1/3 da cotação da pergunta.

2 – Apenas as respostas assinaladas na Folha de Respostas serão consideradas.

3 – Devem ser entregues todas as folhas do exame.

1 - a) ☐ b) ☐ c) ☐ d) ☐

2 - a) ☐ b) ☐ c) ☐ d) ☐

3 - a) ☐ b) ☐ c) ☐ d) ☐

4 - a) ☐ b) ☐ c) ☐ d) ☐

5 - a) ☐ b) ☐ c) ☐ d) ☐

6 - a) ☐ b) ☐ c) ☐ d) ☐

7 - a) ☐ b) ☐ c) ☐ d) ☐

8 - a) ☐ b) ☐ c) ☐ d) ☐

9 - a) ☐ b) ☐ c) ☐ d) ☐

10 - a) ☐ b) ☐ c) ☐ d) ☐

11 - a) ☐ b) ☐ c) ☐ d) ☐

12 - a) ☐ b) ☐ c) ☐ d) ☐

13 - a) ☐ b) ☐ c) ☐ d) ☐

14 - a) ☐ b) ☐ c) ☐ d) ☐

15 - a) ☐ b) ☐ c) ☐ d) ☐

16 - a) ☐ b) ☐ c) ☐ d) ☐

17 - a) ☐ b) ☐ c) ☐ d) ☐

18 - a) ☐ b) ☐ c) ☐ d) ☐

19 - a) ☐ b) ☐ c) ☐ d) ☐

20 - a) ☐ b) ☐ c) ☐ d) ☐

1 – Num sistema computacional as interrupções:

- a) Permitem efetuar chamadas ao sistema operativo e que os periféricos sinalizem a ocorrência de eventos.
- b) Permitem que os periféricos sinalizem a ocorrência de eventos, tal como a finalização de uma operação leitura ou escrita no disco, mas nunca são utilizadas para sinalizar erros no hardware.
- c) Permitem que os periféricos executem chamadas ao sistema operativo.
- d) São utilizadas pelos periféricos, exclusivamente, para iniciar a transferência de grandes quantidades de dados por DMA.

2 – O conjunto seguinte de instruções pode apenas ser executado em kernel mode:

- a) Operações de adição, alteração entre kernel e user mode.
- b) Operações de leitura de uma posição de memória, alteração entre kernel e user mode.
- c) Operações de alteração entre kernel e user mode, acesso às tabelas de interrupção.
- d) Operações de Trap (interrupção de software), alteração entre kernel e user mode.

3 – Relativamente ao funcionamento do barramento de um computador:

- a) O acesso para leitura ou escrita à memória principal implica o gasto de vários ciclos de relógio até concluir a operação.
- b) O acesso para leitura ou escrita à memória principal é sempre feito num ciclo de relógio até concluir a operação.
- c) Não permite que os periféricos acedam à memória principal.
- d) O barramento não interliga o CPU com a memória principal.

4 – A comunicação entre processos através de pipes...

- a) Não permite implementar qualquer tipo de sincronização entre processos..
- b) Implica, sempre, que a função de leitura do pipe seja evocada múltiplas vezes para verificar se o pipe tem dados.
- c) Permite implementar um funcionamento semelhante ao algoritmo dos leitores/escritores uma vez que a função de read não apaga os dados lidos do pipe e a função de write bloqueia se o pipe estiver cheio.
- d) Permite implementar um funcionamento semelhante ao algoritmo de sincronização do produtor/consumidor uma vez que a função de read bloqueia se o pipe estiver vazio e a função de write também bloqueia se o pipe estiver cheio.

5 – Um Cluster de computação ...

- a) É um sistema em que vários processadores estão ligados uns aos outros através de um barramento de alto desempenho.
- b) É um sistema em que os computadores (com um ou mais processadores/cores) estão ligados através de uma rede local de alto desempenho.
- c) É um sistema em que os computadores exclusivamente de baixo desempenho (apenas com um processador e um core) estão ligados através de uma rede local sem fios.
- d) É um sistema em que vários *cores* de um processadores estão ligados uns aos outros através de um barramento de alto desempenho.

6 – O módulo de gestão de I/O:

- a) É responsável pela implementação de operações de comunicação via rede com outros computadores.
- b) Em Linux o módulo denominado de “Virtual File System” fornece uma interface genérica para acesso aos dispositivos de I/O.
- c) Permite atribuir e libertar memória a processos.
- d) Nenhuma das anteriores.

7 – O código seguinte implementa um algoritmo de sincronização entre processos.

P1	P2
1. down (S2) ;	1. down (M) ;
...	2. rc++;
2. Secção crítica	3. if (rc == 2) up(S2) ;
...	4. up (M) ;
3. up (S1) ;	...
	5. Secção crítica
	...
	6. down (M) ;
	7. if (rc == 2) down (S1) ;
	8. up (M) ;

Em que o valor inicial dos semáforos é S1=0, S2=0, M=1 e a variável partilhada rc=0. Considere três processos que irão começar a executar por esta ordem: P2, P1 e P2. O processo P1 não manipula o semáforo M e também não modifica a variável rc.

- a) A secção crítica dos processos é sempre executada, mas sem garantia na ordem de execução.
- b) A secção crítica será sempre executada, primeiro nos processos P2 e só depois no processo P1.
- c) O segundo processo P2 fica sempre bloqueado em deadlock na linha 6.
- d) d) Nenhuma das anteriores.

8 – Relativamente ao escalonamento usando Round Robin, pode-se afirmar o seguinte:

- a) Cada fila tem obrigatoriamente que aplicar o mesmo algoritmo, embora possa aplicar parâmetros diferentes, p.e. utilizar time quantums diferentes entre filas.
- b) Não é uma solução adaptada a sistemas em que se desconhece o comportamento dos processos a escalonar.
- c) Favorece os processos I/O bound.
- d) Nenhuma das anteriores.

9 – Quando uma thread fica bloqueada numa variável de condição depois de chamar a função pthread_cond_wait(...), esta encontra-se no estado de:

- a) Ready
- b) Running
- c) Waiting
- d) Nenhuma das anteriores.

10 – Considere o pseudocódigo seguinte. Assuma que a região de memória já foi criada e inicializada e que podem existir vários processos a executar o programa P1 e o programa P2.

P1	P2	Valores iniciais dos semáforos e memória partilhada
<pre> 1. ... 2. down(a); 3. *x= *x+1; 4. if (*x ==1) 5. down(b); 6. up(a); 7. func_xpto(); 8. down(a); 9. *x= *x-1; 10. if (*x == 0) 11. up(b); 12. up(a); 13. ... </pre>	<pre> 1. ... 2. down(b); 3. func_xpto(); 4. up(b); 5. ... </pre>	<pre> semáforo a=1 semáforo b=1 int *x=0 (memória partilhada, com valor inicial igual a ZERO) </pre>

- Os processos que executam o programa P1 podem executar em paralelo a função `func_xpto()` desde que também exista um processo a executar o programa P2.
- Os processos que executam o programa P1 podem executar em paralelo a função `func_xpto()`, desde que um dos processos que executa o programa P1 já tenha executado com sucesso a linha 5 (`down(b);`).
- Em cada instante, apenas um processo que executa o programa P1 executar em paralelo a função `func_xpto()`, mesmo que não existam processos a executar o programa P2.
- Os processos que executam P1 e P2 não são exclusivos no acesso à invocação da função `func_xpto()`.

11 – Um endereço Lógico:

- É o endereço que indica a posição na memória física onde se encontra um item a ser manipulado.
- É o primeiro endereço de um segmento de memória.
- É o primeiro endereço de uma página de memória.
- É o endereço gerado pelo processador quando um programa está a ser executado antes de chegar à MMU.

12 – Considere um sistema mono-processador cujo sistema operativo utiliza um algoritmo de escalonamento round-robin com *time quantum* igual a 3. Considerando os seguintes perfis de execução para os processos P1, P2 e P3, determine a sequência de execução destes processos num sistema com um processador.

Processo	Perfil do processo	Tempo de chegada
P1	1111-111	0
P2	22-22-222	2
P3	333-33	1

Indique uma possível sequência de execução destes processos, em que um 1, 2 ou 3 representa, respetivamente, o processo P1, P2 ou P3 em execução durante uma unidade de tempo, “-” representa o processo no estado de waiting. Na solução o símbolo “-” significa que o processador não está a executar qualquer processo.

- a) 1113332213322111222
- b) 1111333221113322-222
- c) 1111-111333-3322-22-222
- d) 11133322-1-122-3311222

13 – Assuma o conjunto de processos abaixo, respetivos perfis de execução e um algoritmo de escalonamento round-robin com um *time quantum* de 4, em que a execução é feita utilizando dois processadores, com uma fila global para todos os processos. Indique uma possível sequência de escalonamento para estes processos, assumindo que todos os processos chegam ao sistema no instante 0 (zero).

Processo	Perfil do processo
P1	11---111
P2	2-22--22
P3	33333--3
P4	4---4444

- a) CPU1: 11242111223
CPU2: 333323-4444
- b) CPU1: 3333-22111-3
CPU2: 41124444322-
- c) CPU1: 42--44441113
CPU2: 113333223-22
- d) CPU1: 11422111223
CPU2: 2333334444-

14 – Relativamente aos mecanismos de paginação de memória e segmentação de memória:

- a) Uma página de memória tem sempre o mesmo tamanho para todos os processos.
- b) Uma página de memória pode ter tamanhos diferentes conforme as necessidades dos processos.
- c) Um segmento de memória tem que ter sempre o mesmo tamanho para todos os processos.
- d) Nenhuma das anteriores.

15 – Das seguintes afirmações selecione a opção correta:

- a) Um sistema multimédia não pode ser considerado um sistema de tempo-real.
- b) Um automóvel não é um sistema de tempo-real.
- c) O escalonamento Round-Robin é adequado para sistemas de tempo-real.
- d) Num sistema de tempo-real, a *deadline* é o instante no tempo até ao qual o resultado de uma computação deve estar disponível.

16 – Considere o pseudocódigo seguinte, referente às threads T1 e T2. Assuma que existem duas variáveis de exclusão mútua m1 e m2, que são acessíveis a todas as threads e já foram inicializadas. Considerando o código executado pelas threads T1 e T2 e que as funções utilizadas seguem o seu funcionamento normal, sem erros:

T1	P2
<pre>... pthread_mutex_lock(&m1); /* Executa bloco A */ pthread_mutex_lock(&m1); /* Executa bloco C */ pthread_mutex_unlock(&m2); pthread_mutex_unlock(&m1);</pre>	<pre>... pthread_mutex_lock(&m2); /* Executa bloco B */ pthread_mutex_unlock(&m1); pthread_mutex_lock(&m2); /* Executa bloco D */ pthread_mutex_unlock(&m2);</pre>

Indique a resposta correta.

- A utilização das variáveis de exclusão mútua assegura que o bloco A é sempre executado primeiro que o bloco B
- O bloco C é sempre executado antes do bloco D pois a thread T1 só faz o unlock da variável de exclusão mútua m2 após a execução do bloco C
- O código apresentado não respeita o funcionamento e utilização das variáveis de exclusão mútua, já que só a thread que efetuou o lock é que poderá efetuar o unlock da mesma variável de exclusão
- Para uma correta implementação da solução é necessário utilizar uma terceira variável de exclusão mútua na thread T1 antes da execução do bloco C

17 – Considere o programa seguinte:

```
int main()
{
    if (fork() == 0){
        printf("C");
        fflush(stdout);
    }
    if (fork() == 0){
        printf("A");
        fflush(stdout);
    }
    else
    {
        printf("B");
        fflush(stdout);
    }
    exit(0);
}
```

Um possível output será:

- BAACB
- BCABA
- AACBB
- BABAC

18 – A lei de Amdhal define o máximo speedup que se pode obter para um programa ao proceder à sua paralelização. O speedup depende ...

- Apenas da percentagem do programa que não pode ser paralelizada e do número de processadores.
- Apenas da percentagem do programa que se pode paralelizar e do número de processadores.
- Apenas da percentagem do programa que não pode ser paralelizada, percentagem do programa que se pode paralelizar e do número de processadores.
- Nenhuma das anteriores.

19 – Considere o pseudocódigo seguinte. Assuma que a região de memória já foi criada e inicializada e que podem ser executados vários processos a executar o programa P1 e o programa P2.

Solução 1	Solução 2
<pre> ... public class solucao1 { private long a=100; private Object acesso = new Object(); ... public void metodo_alt(){ ... synchronized(acesso){ //zona critica // alterar o valor da variável a ... } ... } } </pre>	<pre> ... public class solucao2 { private long a=100; private Object acesso = new Object(); ... public synchronized void metodo_alt(){ ... //zona critica //alterar o valor da variável a ... } } </pre>

Indique a resposta correta.

- a) A solução 1 não garante a coerência dos dados devido a não utilizar synchronized antes do nome do método metodo_alt().
- b) As duas soluções garantem a coerência dos dados. A solução 1 permite uma sincronização mais fina e por esse motivo aumenta a probabilidade de uma execução concorrente mais otimizada.
- c) A solução 1 permite que mais do que uma thread acesse à zona crítica em simultâneo
- d) A solução 1 não funciona pois um objeto apenas pode ser utilizado por uma thread

20

21 – Considere o extrato de código seguinte em OpenMP:

```

...
#pragma omp parallel num_threads(2)
{
    printf("T1\n");
    if ( omp_get_thread_num() == 2 ) {
        printf("T2");
    }
}
...

```

- a) Vai criar 2 threads, ambas imprimem "T1", apenas a thread 2 vai imprimir "T2".
- b) Vai criar tantas threads como o número de cores/processadores disponíveis, todas imprimem "T1", apenas a thread 2 vai imprimir "T2".
- c) Vai criar 4 threads, todas imprimem "T1", apenas a thread 2 vai imprimir "T2".
- d) Não vai criar nenhuma thread.